Software Maintenance
23B61CA221
BCA, 4th sem

Software Maintenance

What is Software Maintenance?

Software Maintenance is a very broad activity that includes error corrections, enhancements of capabilities, deletion of obsolete capabilities, and optimization.

Why?

1. Addressing Bugs and Errors:
2. Enhancing Features and Performance:
3. Adapting to Changing Requirements:
4. Enhancing Security:
5. Extending Software Lifespan:
6. Cost Efficiency:
7. Ensuring Compliance:

Software Maintenance

Categories of Maintenance

- Corrective maintenance

This refer to modifications initiated by defects in the software.

Fix bugs or defects discovered after the software is released

- Adaptive maintenance

It includes modifying the software to match changes in the ever changing environment. (e.g., new OS, hardware, or software dependencies).

- Perfective maintenance

It means improving processing efficiency or performance, or restructuring the software to improve changeability. This may include enhancement of existing system functionality, improvement in computational efficiency etc.

Adding new features or improving the user interface.

Software Maintenance

- Other types of maintenance — Preventive Maintenance

There are long term effects of corrective, adaptive and perfective changes. This leads to increase in the complexity of the software, which reflect deteriorating structure. The work is required to be done to maintain it or to reduce it, if possible. This work may be named as preventive maintenance.

Make changes to prevent future problems or to improve the software's long-term stability and maintainability.

Refactoring code or updating documentation to reduce future maintenance effort.

Software Maintenance

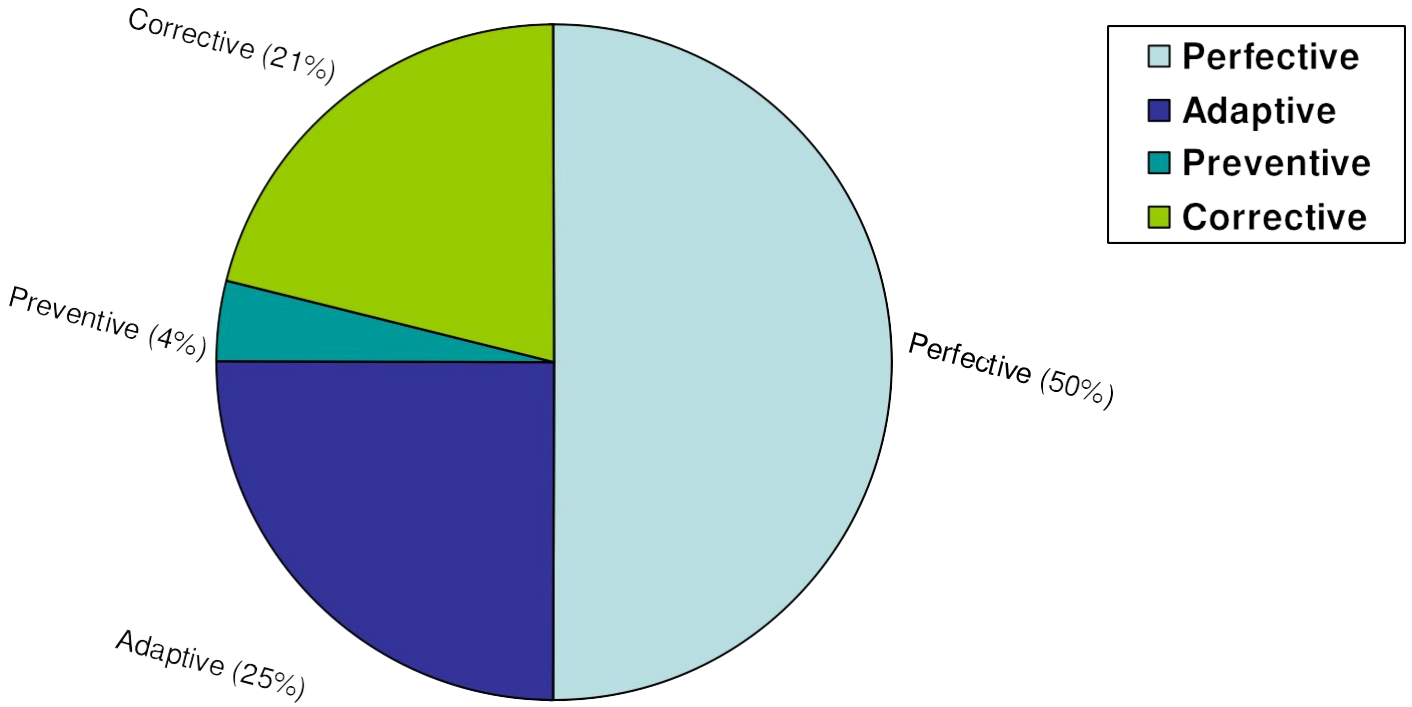

Fig. 1: Distribution of maintenance effort

Software Maintenance

Problems During Maintenance

- Often the program is written by another person or group of persons.
- Often the program is changed by person who did not understand it clearly.
- Program listings are not structured.
- High staff turnover.
- Information gap.
- Systems are not designed for change.

Potential Solutions to Maintenance Problems

- Budget and effort reallocation
- Complete replacement of the system
- Maintenance of existing system

Software Maintenance

The Maintenance Process

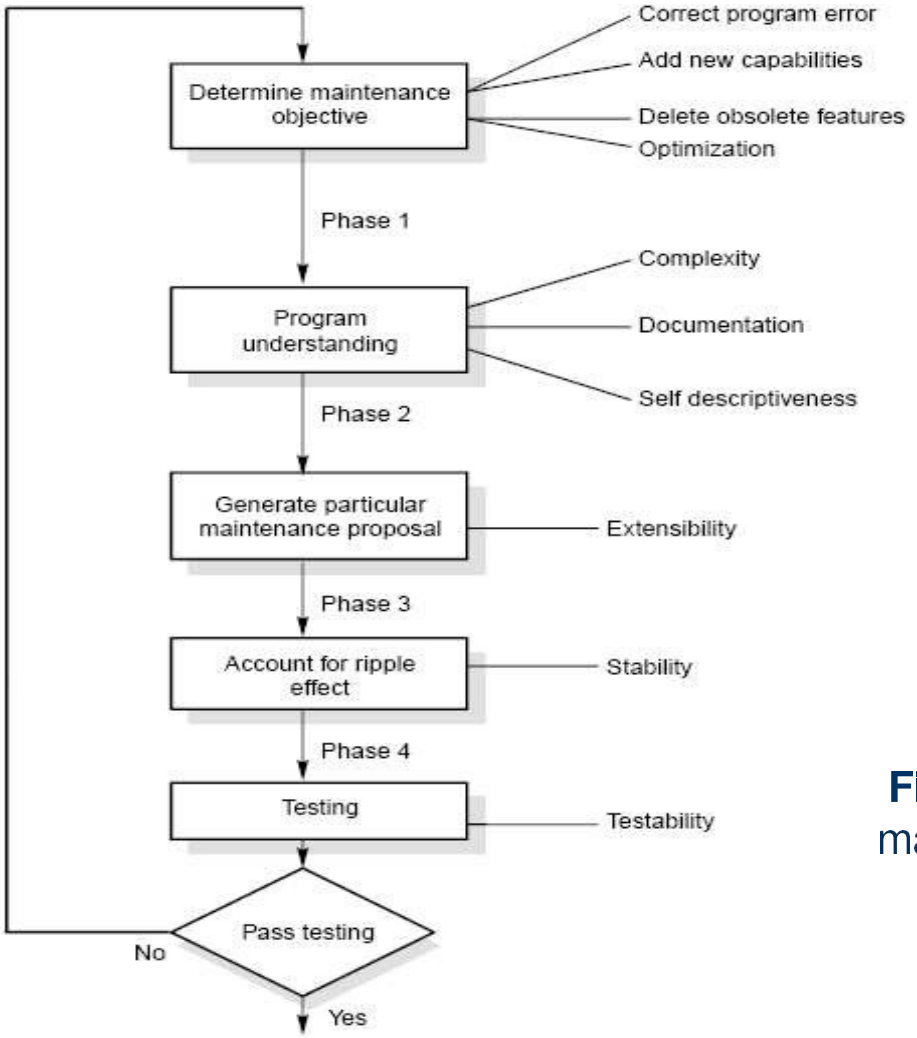

Fig. 2: The software maintenance process

Software Maintenance

- Program Understanding

The first phase consists of analyzing the program in order to understand.

- Generating Particular Maintenance Proposal

The second phase consists of generating a particular maintenance proposal to accomplish the implementation of the maintenance objective.

- Ripple Effect

The third phase consists of accounting for all of the ripple effect as a consequence of program modifications.

Software Maintenance

- Modified Program Testing

The fourth phase consists of testing the modified program to ensure that the modified program has at least the same reliability level as before.

- Maintainability

Each of these four phases and their associated software quality attributes are critical to the maintenance process. All of these factors must be combined to form maintainability.

Maintenance Models

- Quick-fix Model

This is basically an adhoc approach to maintaining software. It is a fire fighting approach, waiting for the problem to occur and then trying to fix it as quickly as possible.

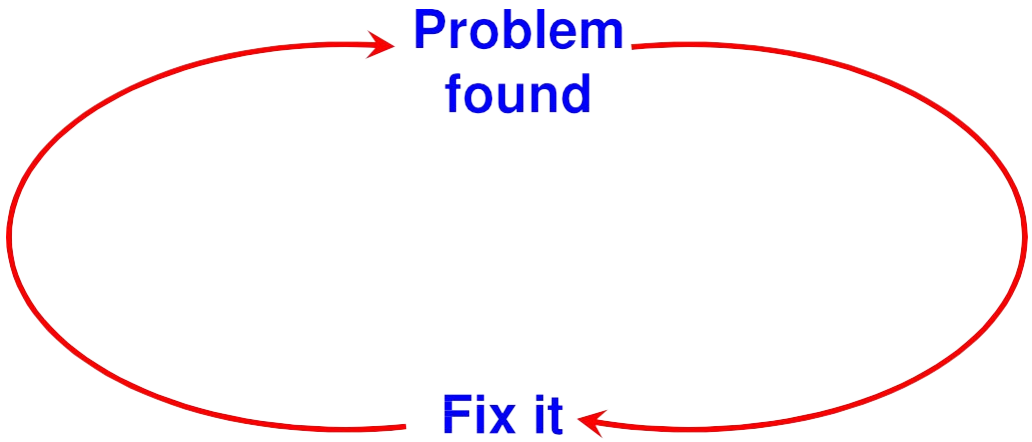

Fig. 3: The quick-fix model

Software Maintenance

- Iterative Enhancement Model
  - Analysis
  - Characterization of proposed modifications
  - Redesign and implementation

Software Maintenance

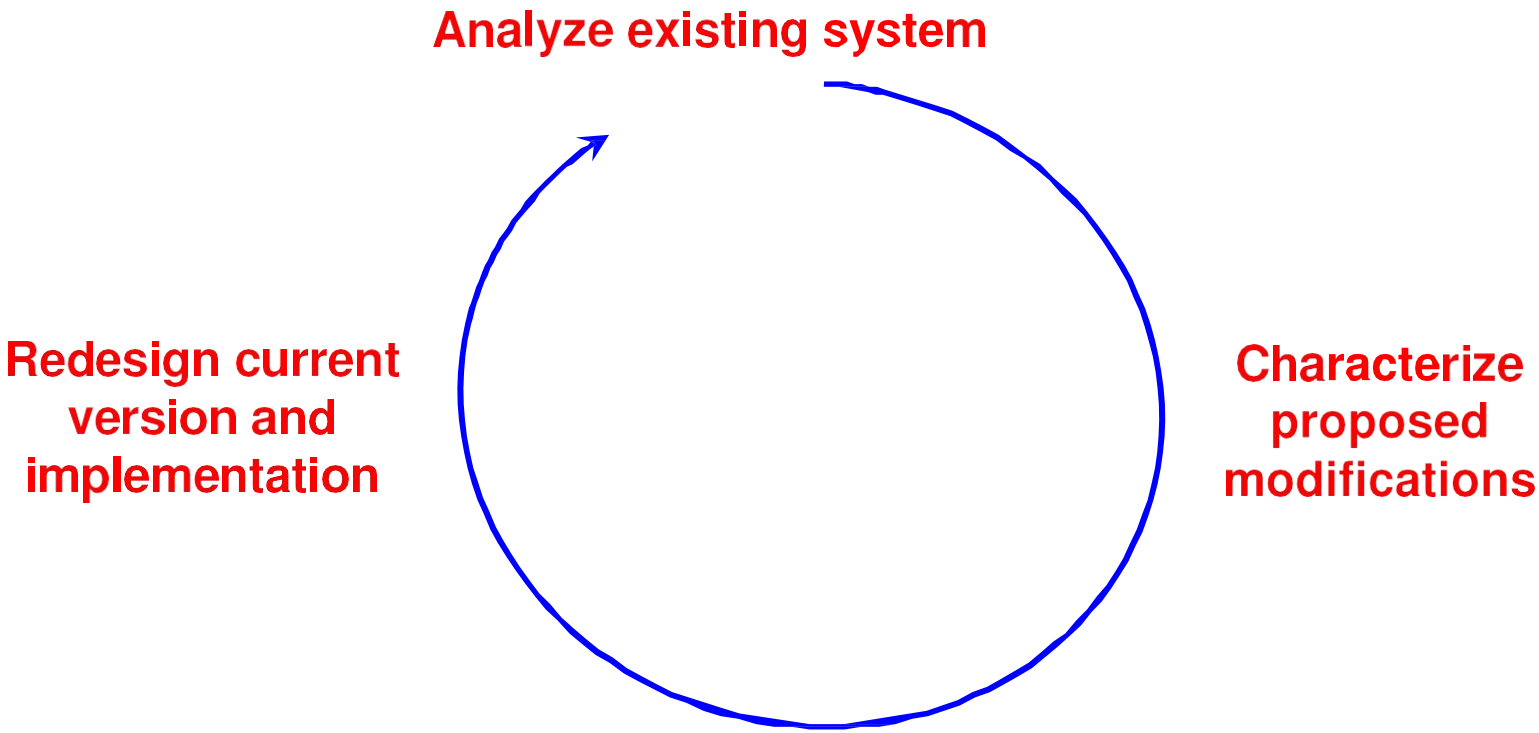

Fig. 4: The three stage cycle of iterative enhancement

Software Maintenance

- Reuse Oriented Model

The reuse model has four main steps:

1. Identification of the parts of the old system that are candidates for reuse.
2. Understanding these system parts.
3. Modification of the old system parts appropriate to the new requirements.
4. Integration of the modified parts into the new system.

Software Maintenance

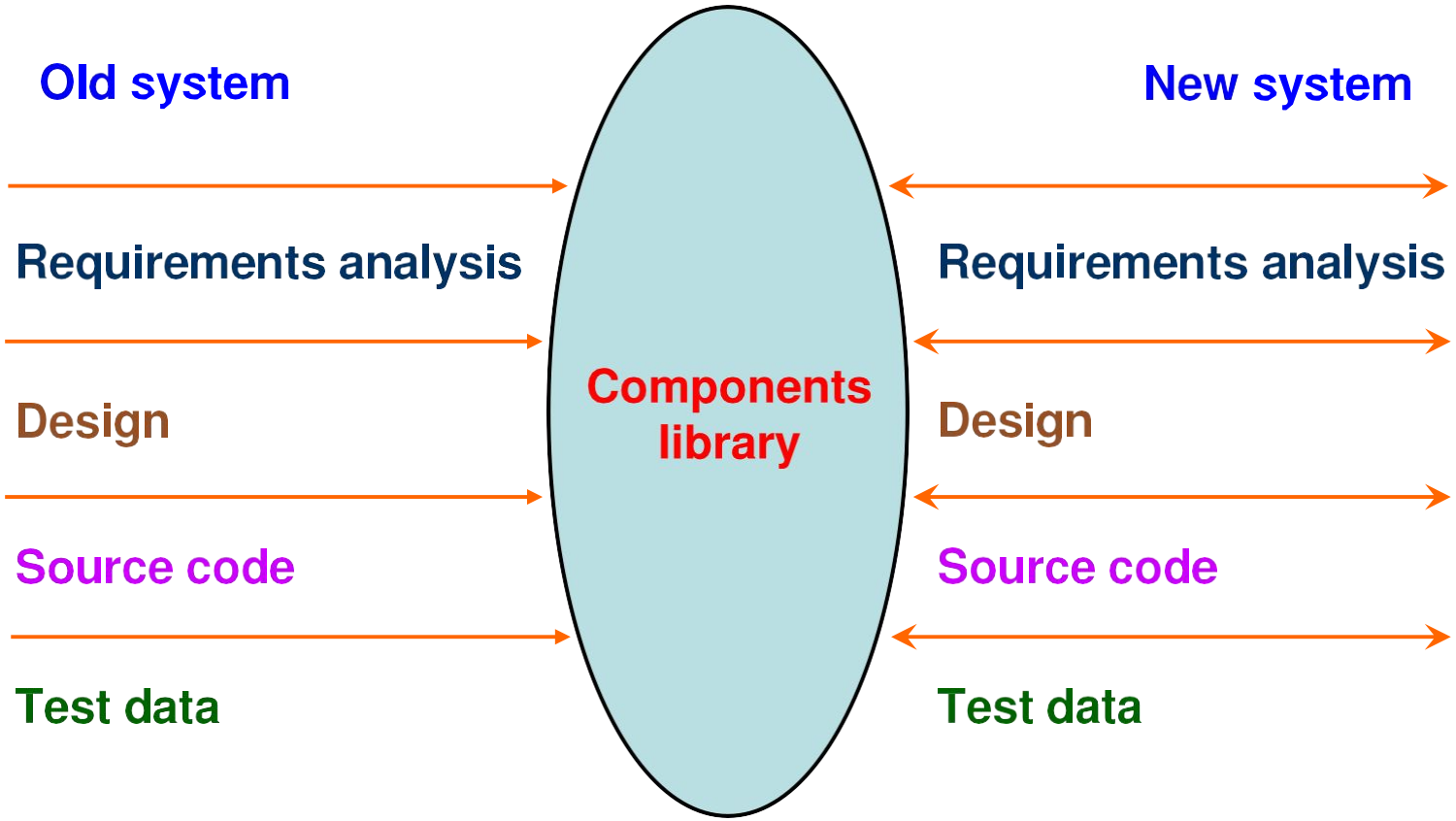

Fig. 5: The reuse model

Software Maintenance

- Boehm's Model

Boehm proposed a model for the maintenance process based upon the economic models and principles.

Boehm represent the maintenance process as a closed loop cycle.

Software Maintenance

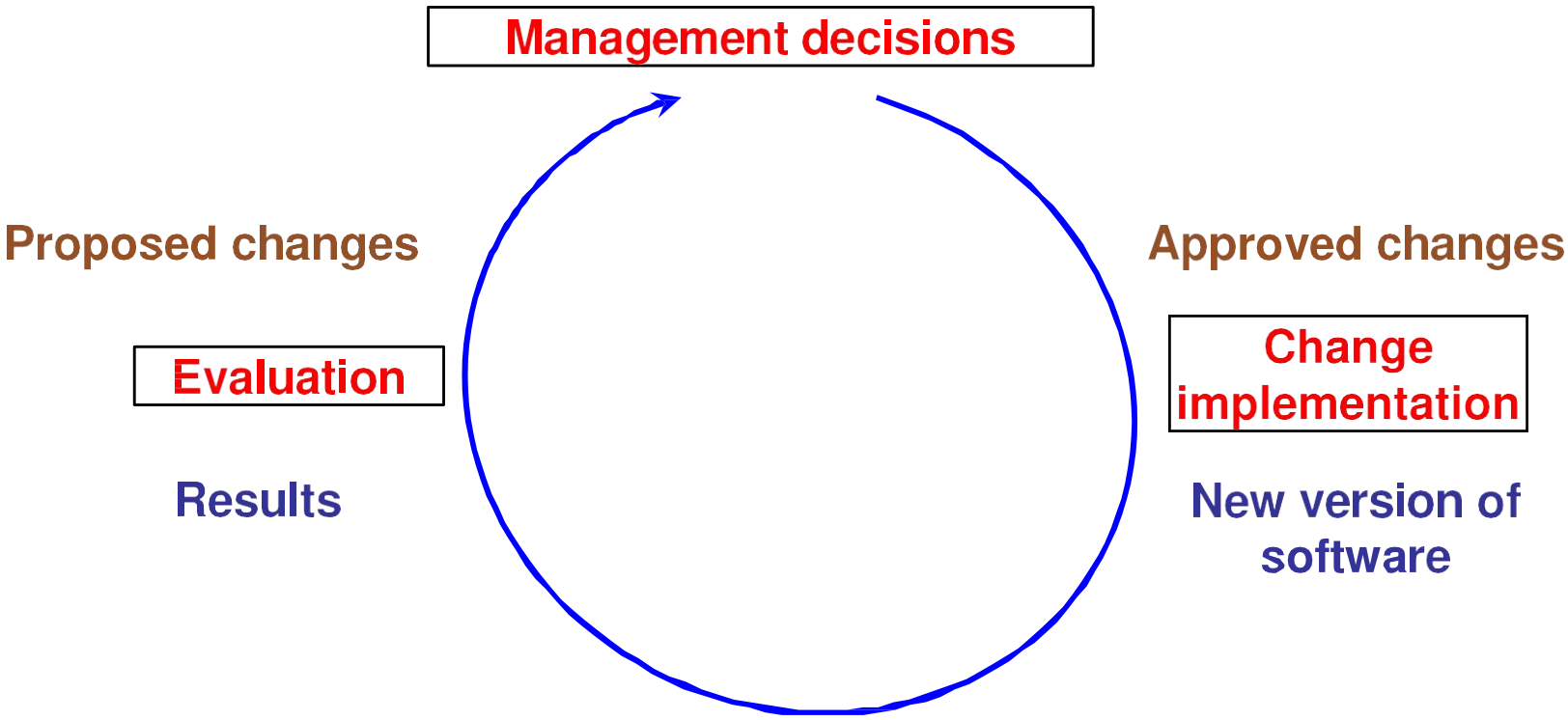

Fig. 6: Boehm's model

Software Maintenance

- Taute Maintenance Model

It is a typical maintenance model and has eight phases in cycle fashion. The phases are shown in Fig. 7

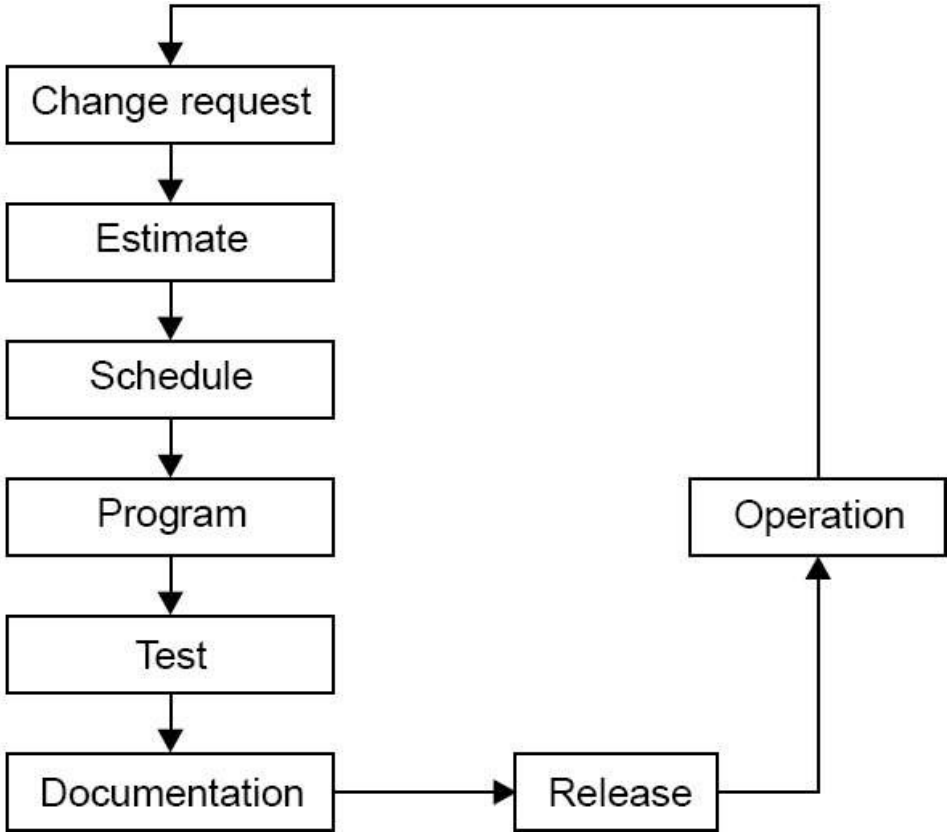

Fig. 7: Taute maintenance model

Software Maintenance

Phases :

1. Change request phase
2. Estimate phase
3. Schedule phase
4. Programming phase
5. Test phase
6. Documentation phase
7. Release phase
8. Operation phase

Estimation of maintenance costs

Phase	Ratio
Analysis	1
Design	10
Implementation	100

Table 3: Defect repair ratio

Regression Testing

Regression testing is the process of retesting the modified parts of the software and ensuring that no new errors have been introduced into previously test code.

"Regression testing tests both the modified code and other parts of the program that may be affected by the program change. It serves many purposes :

- increase confidence in the correctness of the modified program
- locate errors in the modified program
- preserve the quality and reliability of software
- ensure the software's continued operation

Software Maintenance

- Development Testing Versus Regression Testing

Sr. No.	Development testing	Regression testing
1.	We create test suites and test plans	We can make use of existing test suite and test plans
2.	We test all software components	We retest affected components that have been modified by modifications.
3.	Budget gives time for testing	Budget often does not give time for regression testing.
4.	We perform testing just once on a software product	We perform regression testing many times over the life of the software product.
5.	Performed under the pressure of release date of the software	Performed in crisis situations, under greater time constraints.

- Regression Test Selection

Regression testing is very expensive activity and consumes significant amount of effort / cost. Many techniques are available to reduce this effort/ cost.

1. Reuse the whole test suite

2. Reuse the existing test suite, but to apply a regression test selection technique to select an appropriate subset of the test suite to be run.

Fragment A		Fragment B (modified form of A)	
S_1	y = (x - 1) * (x + 1)	S_1'	y = (x -1) * (x + 1)
S_2	if (y = 0)	S_2'	if (y = 0)
S_3	return (error)	S_3'	return (error)
S_4	else	S_4'	else
S_5	return $\left(\frac{1}{y}\right)$	S_5'	return $\left(\frac{1}{y-3}\right)$

Fig. 8: code fragment A and B

Test cases		
Test number	Input	Execution History
t_1	x = 1	S_1, S_2, S_3
t_2	x = -1	S_1, S_2, S_3
t_3	x = 2	S_1, S_2, S_5
t_4	x = 0	S_1, S_2, S_5

Fig. 9: Test cases for code fragment A of Fig. 8

If we execute all test cases, we will detect this divide by zero fault. But we have to minimize the test suite. From the fig. 9, it is clear that test cases t_3 and t_4 have the same execution history i.e. S_1, S_2, S_5. If few test cases have the same execution history; minimization methods select only one test case. Hence, either t_3 or t_4 will be selected. If we select t_4 then fine otherwise fault not found.

Minimization methods can omit some test cases that might expose fault in the modified software and so, they are not safe.

A safe regression test selection technique is one that, under certain assumptions, selects every test case from the original test suite that can expose faults in the modified program.

Reverse Engineering

Reverse engineering is the process followed in order to find difficult, unknown and hidden information about a software system.

Software Maintenance

- Scope and Tasks

The areas there reverse engineering is applicable include (but not limited to):

1. Program comprehension
2. Redocumentation and/ or document generation
3. Recovery of design approach and design details at any level of abstraction
4. Identifying reusable components
5. Identifying components that need restructuring
6. Recovering business rules, and
7. Understanding high level system description

Software Maintenance

Reverse Engineering encompasses a wide array of tasks related to understanding and modifying software system. This array of tasks can be broken into a number of classes.

- Mapping between application and program domains

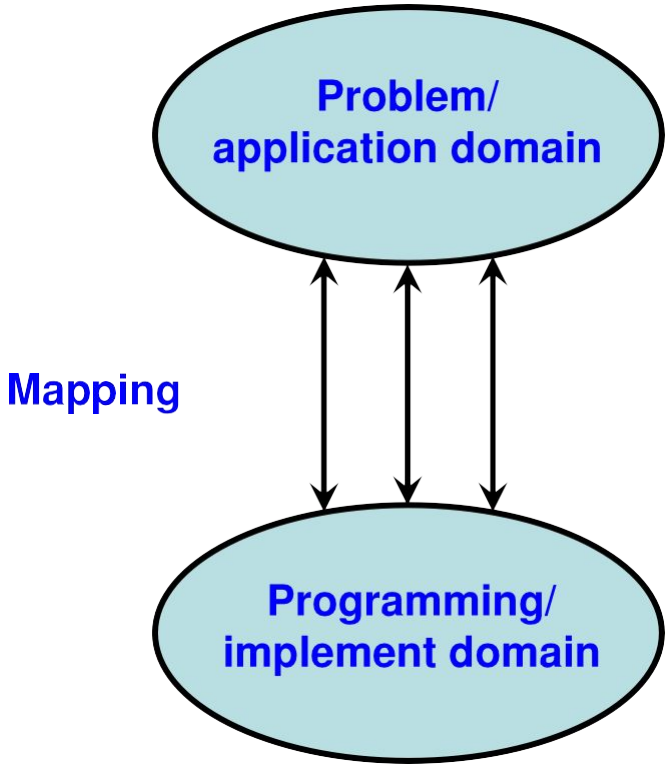

Fig. 10: Mapping between application and domains program

Software Maintenance

- Mapping between concrete and abstract levels
- Rediscovering high level structures
- Finding missing links between program syntax and semantics
- To extract reusable component

Software Maintenance

- Levels of Reverse Engineering

Reverse Engineers detect low level implementation constructs and replace them with their high level counterparts.

The process eventually results in an incremental formation of an overall architecture of the program.

Software Maintenance

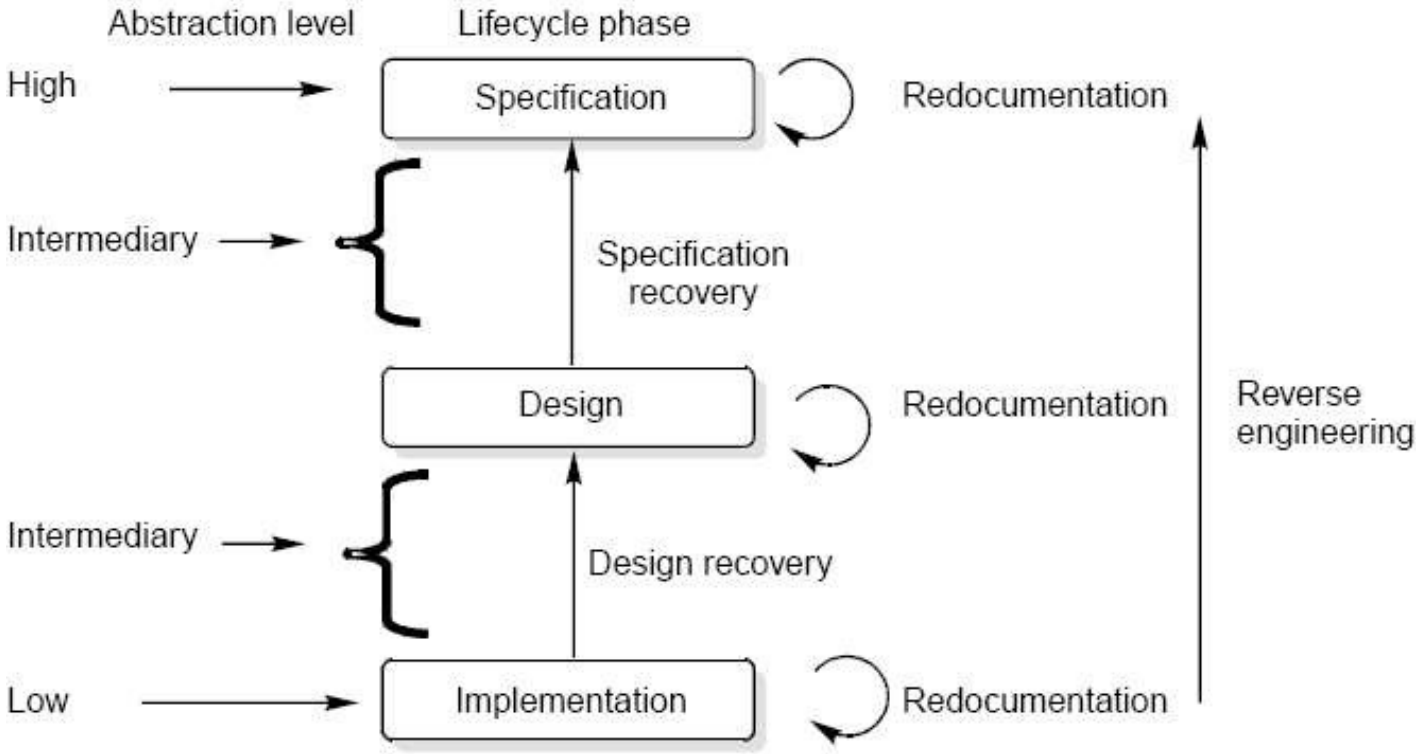

Fig. 11: Levels of abstraction

Software Maintenance

Redocumentation

Redocumentation is the recreation of a semantically equivalent representation within the same relative abstraction level.

Design recovery

Design recovery entails identifying and extracting meaningful higher level abstractions beyond those obtained directly from examination of the source code. This may be achieved from a combination of code, existing design documentation, personal experience, and knowledge of the problem and application domains.

Software RE-Engineering

Software re-engineering is concerned with taking existing legacy systems and re-implementing them to make them more maintainable.

The critical distinction between re-engineering and new software development is the starting point for the development as shown in Fig.12.

Software Maintenance

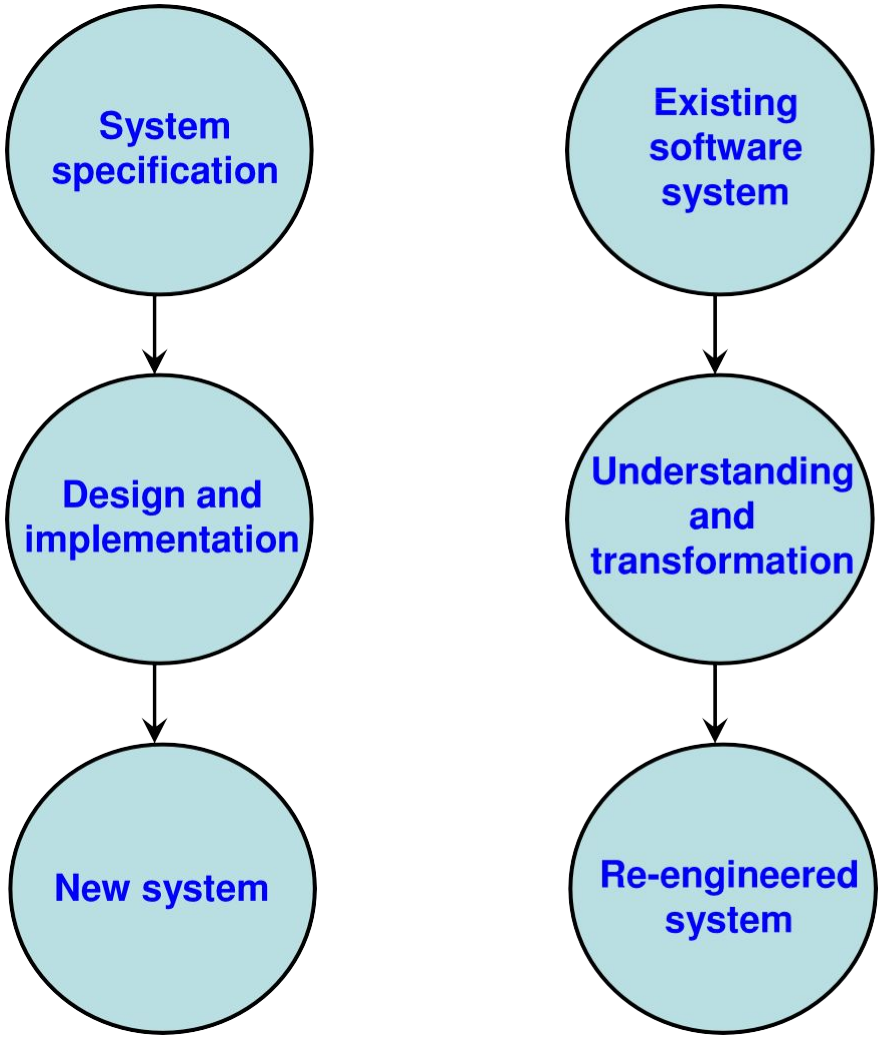

Fig. 12: Comparison of new software development with re-engineering

Software Maintenance

The following suggestions may be useful for the modification of the legacy code:

- ✓ Study code well before attempting changes
- ✓ Concentrate on overall control flow and not coding
- ✓ Heavily comment internal code
- ✓ Create Cross References
- ✓ Build Symbol tables
- ✓ Use own variables, constants and declarations to localize the effect
- ✓ Keep detailed maintenance document
- ✓ Use modern design techniques